WEB APPLICATION PENETRATION TESTING

bWAPP — Buggy Web Application

Graduation Project — Security Assessment Report

NTI – (National Telecommunication Institute)

OUR TEAM — nullSEC

Penetration Testers & Security Researchers

Mohammed Ashraf Mokhtar Elsayed — *Team Leader, Penetration Tester & Security Researcher*

Mohamed Ismail Abdallah Ismail — *Penetration Tester & Security Researcher*

Mohamed Ahmed Albasiony — *Penetration Tester & Security Researcher*

Rana Ayman Elsayed Ahmed — *Penetration Tester & Security Researcher*

Mohammed Ashraf Abdelwahab — *Penetration Tester & Security Researcher*

Target	bWAPP (Buggy Web Application), Version 2.2
Assessment Type	Black-box / Grey-box Web Application Penetration Testing
Environment	Local bWAPP instance and hosted training instance documented by the team
Security Level	Low, unless otherwise stated
Primary Methodology	OWASP Web Security Testing Guide (WSTG)
Assessment Guide	NIST SP 800-115
Date	August 2026

1. Executive Summary

This report presents the results of a controlled web application penetration test against bWAPP (Buggy Web Application), a deliberately vulnerable application designed for security training. The objective was to identify, validate, and document exploitable weaknesses through a structured testing process.

The assessment was organized using the OWASP Web Security Testing Guide as the primary web-application testing reference and NIST SP 800-115 as the technical assessment guide. The workflow was structured into reconnaissance, scanning and vulnerability identification, exploitation/validation, impact assessment, and reporting.

Six vulnerabilities were documented from the team's bWAPP testing evidence: Union-based SQL Injection, Remote/Local File Inclusion, Reflected XSS through PHP_SELF, Reflected XSS through POST, IDOR/client-side price tampering, and Reflected HTML Injection.

The most significant issues were SQL Injection and RFI/LFI, both rated Critical in the original assessment. These weaknesses demonstrate that untrusted client input can reach sensitive database queries and file-inclusion functionality without adequate server-side validation.

1.1 Key Findings

ID	Finding	Severity	OWASP WSTG Area	Tester
F-01	SQL Injection — Union-based	**Critical**	Input Validation	Mohamed Ismail
F-02	Remote & Local File Inclusion	**Critical**	Input Validation / File Inclusion	Mohammed Ashraf Mokhtar
F-03	Reflected XSS — PHP_SELF / GET	**High**	Input Validation	Mohammed Ashraf Mokhtar
F-04	Reflected XSS — POST	**Medium**	Input Validation	Rana Ayman
F-05	IDOR / Price Tampering	**High**	Authorization / Business Logic	Mohammed Ashraf Abdelwahab
F-06	HTML Injection — Reflected GET	**Medium**	Input Validation	Mohamed Ahmed Albasiony

2. Engagement Scope

2.1 Target and Environment

- Application under test: bWAPP (Buggy Web Application), version 2.2.
- The team's report documents a local instance at 127.0.0.1/bWAPP and a hosted training instance at bwapp.hakhub.net.
- The application was tested at Low security level unless a finding states otherwise.
- Testing was limited to the intentionally vulnerable training application and its exposed web functionality.

2.2 Objectives

1. Identify application-level attack surfaces and security weaknesses.
2. Validate suspected vulnerabilities using controlled proof-of-concept requests.
3. Assess the security impact of confirmed vulnerabilities.
4. Map the testing activities to recognized web-security testing guidance.
5. Produce reproducible evidence and practical remediation recommendations.

2.3 Team Responsibilities

Team Member	Assigned Area
Mohamed Ahmed Albasiony	Reflected HTML Injection
Mohamed Ismail	Union-based SQL Injection
Mohammed Ashraf Mokhtar	Reflected XSS (PHP_SELF) and RFI/LFI
Rana Ayman	Reflected XSS (POST)
Mohammed Ashraf Abdelwahab	IDOR / client-side price tampering

3. Methodology & Security Testing Framework

The assessment was not treated as a collection of isolated vulnerability demonstrations. The report follows a structured penetration-testing workflow so that the reader can see how the team moved from discovering the application attack surface to validating vulnerabilities and documenting remediation.

3.1 Primary Framework — OWASP Web Security Testing Guide (WSTG)

The OWASP Web Security Testing Guide (WSTG) is used as the primary application-security testing reference. OWASP organizes web testing into areas such as Information Gathering, Configuration and Deployment Management, Identity Management, Authentication, Authorization, Session Management, Input Validation, Error Handling, Business Logic, and Client-side Testing.

For this bWAPP assessment, the relevant WSTG areas were selected according to the functionality actually tested. The report therefore uses OWASP as a testing framework rather than simply listing the OWASP Top 10 as a checklist.

Assessment Activity	OWASP WSTG Area	How Applied in bWAPP
Application discovery	Information Gathering	Identify accessible bWAPP pages, forms, parameters and application behaviour.
Input manipulation	Input Validation Testing	Test user-controlled parameters for SQLi, XSS, file inclusion and HTML injection.
Access/business-logic testing	Authorization / Business Logic	Test whether client-controlled object or pricing values are trusted by the server.
Evidence and reporting	Reporting	Document reproduction steps, evidence, impact and remediation.

3.2 NIST SP 800-115

NIST SP 800-115, Technical Guide to Information Security Testing and Assessment, is used as the assessment and testing-process reference. It provides practical guidance for planning and conducting technical security tests, analyzing findings, and developing mitigation strategies.

In this project, NIST is used to structure the engagement around discovery/testing, validation and analysis, and reporting. OWASP supplies the web-application-specific test areas, while NIST provides the broader assessment process.

3.3 Testing Phases

Phase	Purpose	Main Activities	Output
1. Reconnaissance	Understand the target and attack surface	Target identification, application mapping, technology/parameter observation	Attack-surface map
2. Scanning & Identification	Locate potential weaknesses	Request analysis, input probing, parameter testing	Suspected vulnerabilities
3. Exploitation / Validation	Prove exploitability safely	Burp Proxy/Repeater, crafted payloads, controlled PoCs	Confirmed vulnerabilities

4. Impact Analysis	Determine security consequences	Data exposure, script execution, file disclosure, business-logic impact	Risk/severity
5. Reporting	Communicate evidence and fixes	Screenshots, reproduction steps, remediation	Final penetration-test report

3.4 Tools Used

- Burp Suite — Proxy/Intercept and Repeater for HTTP request inspection and controlled modification.
- Browser developer tools — observing application behaviour and client-side execution.
- Kali Linux — testing environment.
- Manual payload construction — SQL Injection, XSS, path traversal/file inclusion, HTML injection and parameter tampering.

4. Reconnaissance

Reconnaissance is the first technical phase of the assessment. The objective is to understand the target, identify reachable application components, discover input points, and establish a baseline before attempting exploitation.

Important scope note: the provided bWAPP evidence is primarily application-level. Unlike the separate Final Web report, it does not contain documented Nmap/subdomain-enumeration results. Therefore, this section does not claim external asset-discovery results that are not present in the bWAPP evidence. The reconnaissance documented here is limited to what the team's bWAPP material supports.

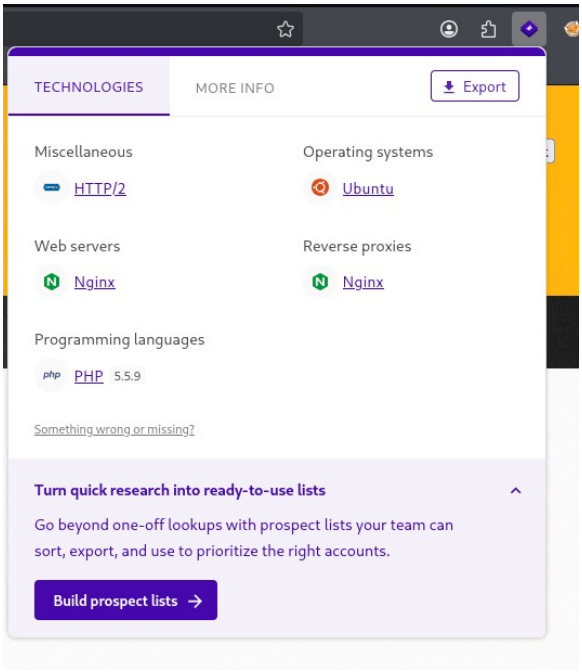
4.1 Target Identification

Application	bWAPP — Buggy Web Application
Version	2.2
Documented local target	127.0.0.1/bWAPP
Documented hosted target	bwapp.hakhub.net
Security level	Low

The target was identified as a deliberately vulnerable web application. The assessment therefore focused on discovering and testing application functionality and its input parameters rather than performing broad internet reconnaissance.

4.2 Technology Fingerprinting

Passive technology fingerprinting was performed to profile the target's underlying stack before functional testing began. This was carried out using Wappalyzer, which inspects HTTP response headers, page markup and other observable artefacts to identify server-side and infrastructure components without sending intrusive requests. The results below establish the software baseline used to scope subsequent vulnerability and exploit research (for example, confirming that PHP-specific injection and file-inclusion techniques were an appropriate line of attack).



Wappalyzer technology fingerprint of the bWAPP target

Category	Component	Relevance to Assessment
Operating system	Ubuntu	Confirms the Linux-based hosting environment for the application.
Web server	Nginx	Identifies the front-facing server handling HTTP requests.
Reverse proxy	Nginx	Indicates request routing/termination ahead of the application, relevant when interpreting response headers and caching behaviour.
Programming language	PHP 5.5.9	An end-of-life PHP version, consistent with the intentionally vulnerable, unpatched nature of the target and directly relevant to the SQL Injection and RFI/LFI findings.
Protocol	HTTP/2	Confirms the transport-layer capability exposed by the front-end server.

Taken together, these results describe an outdated, unpatched LAMP-style stack fronted by Nginx — consistent with a deliberately vulnerable training target and supportive of the injection and inclusion vectors pursued later in the assessment.

4.3 Application Mapping

The team mapped individual bWAPP modules by navigating through the application and identifying pages that accept user-controlled input. The documented findings show multiple attack surfaces, including movie search, language/file selection, reflected input forms, ticket ordering, and HTML-rendering functionality.

Application Function	Endpoint	Interesting Input
Movie search	sqli_6.php	title
Language/file selection	rlfi.php	language

Reflected XSS — PHP_SELF	xss_php_self.php	firstname / lastname
Reflected XSS — POST	xss_post.php	firstname / lastname
Ticket ordering	insecure_direct_object_ref_2.php	ticket_quantity / ticket_price
HTML Injection — GET	htmli_get.php	firstname / lastname

4.4 Baseline Behaviour

Before injecting malicious input, normal values were submitted to understand expected application behaviour. This baseline was important for distinguishing normal responses from security-relevant changes. For example, normal search values were compared with UNION-based SQLi responses, normal language values with file-inclusion attempts, and normal ticket prices with tampered price values.

4.5 Parameter Discovery and Input Identification

During application mapping, parameters that influence server-side processing or page output were identified. These parameters became candidates for further testing because they represent trust boundaries between the browser and the server.

- title — used by the movie-search functionality.
- language — used to select included language/file content.
- firstname / lastname — reflected into application responses.
- ticket_quantity / ticket_price — submitted during ticket ordering.

4.6 Reconnaissance Outcome

The reconnaissance phase produced an application attack-surface map that was subsequently used to prioritize input-validation, file-inclusion, reflected-output, authorization/business-logic and injection tests.

Discovered Area	Potential Test	Resulting Finding
Search input	SQL injection	F-01 SQL Injection
File/language input	File inclusion / traversal	F-02 RFI/LFI
Reflected name inputs	Reflected script injection	F-03 / F-04 XSS
Order parameters	Client-side trust / authorization logic	F-05 IDOR / price tampering
Reflected HTML inputs	HTML injection	F-06 HTML Injection

5. Scanning & Vulnerability Identification

After reconnaissance, the team moved from discovery to focused vulnerability identification. Because bWAPP is intentionally vulnerable, the emphasis was on manual validation rather than treating automated scanner output as proof.

5.1 HTTP Request Analysis

- Intercept normal requests with Burp Suite.
- Identify parameters controlled by the client.
- Change one parameter at a time to isolate the application's trust boundary.
- Compare the modified response with the baseline response.
- Record the request and response as evidence when a security impact is confirmed.

5.2 Vulnerability Identification Strategy

Test Class	Entry Point	Validation Technique	Evidence
SQL Injection	title	Quote manipulation → UNION-based payload	Database data displayed in response
File Inclusion	language	Remote URL and ../ traversal	Remote content and /etc/passwd disclosure
Reflected XSS	firstname / lastname	Script payload in GET/POST	JavaScript execution / cookie exposure
IDOR / business logic	ticket parameters	Modify client-supplied values	Incorrect order price accepted
HTML Injection	firstname / lastname	HTML markup payload	Injected heading/link rendered

5.3 Validation Principle

A suspected vulnerability was treated as confirmed only when the modified input caused a security-relevant response that could be reproduced. This distinction is important because a scanner finding or unusual response alone does not establish exploitability.

6. Detailed Findings

Each finding follows the same structure: description, affected location, severity, OWASP mapping, reproduction procedure, evidence, impact, and remediation.

6.1 SQL Injection — Union-Based

Finding ID	F-01
Severity	**Critical — CVSS 3.1: 9.8**
Location	sqli_6.php — title parameter
Tester	Mohamed Ismail
OWASP Mapping	WSTG Input Validation Testing — Testing for SQL Injection (WSTG-INPV-05)

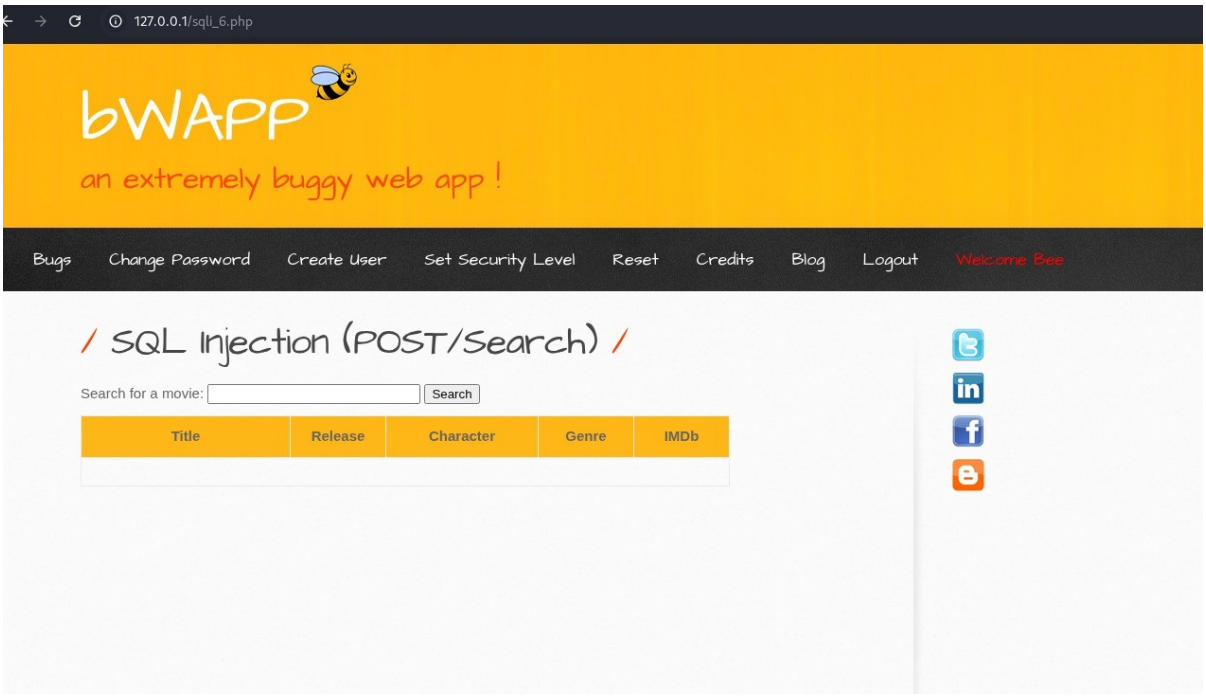
Description

The movie-search functionality accepts the title parameter and uses it in a database query without adequate parameterisation. A UNION-based payload was used to alter the query and return data from the users table.

Steps to Reproduce

1. Submit a normal movie-search request to establish the baseline.
2. Intercept the POST request to sqli_6.php using Burp Suite.
3. Send the request to Repeater and modify the title parameter.
4. Use the documented UNION-based proof of concept: 0' union select all 1, concat(id,login),password,email,secret,6,7 from users #.
5. Forward the request and observe user-related data in the application response.

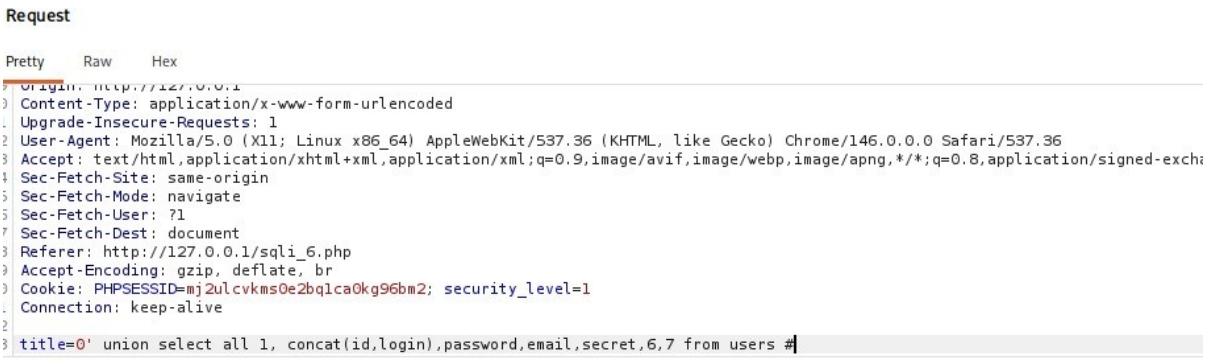
Evidence



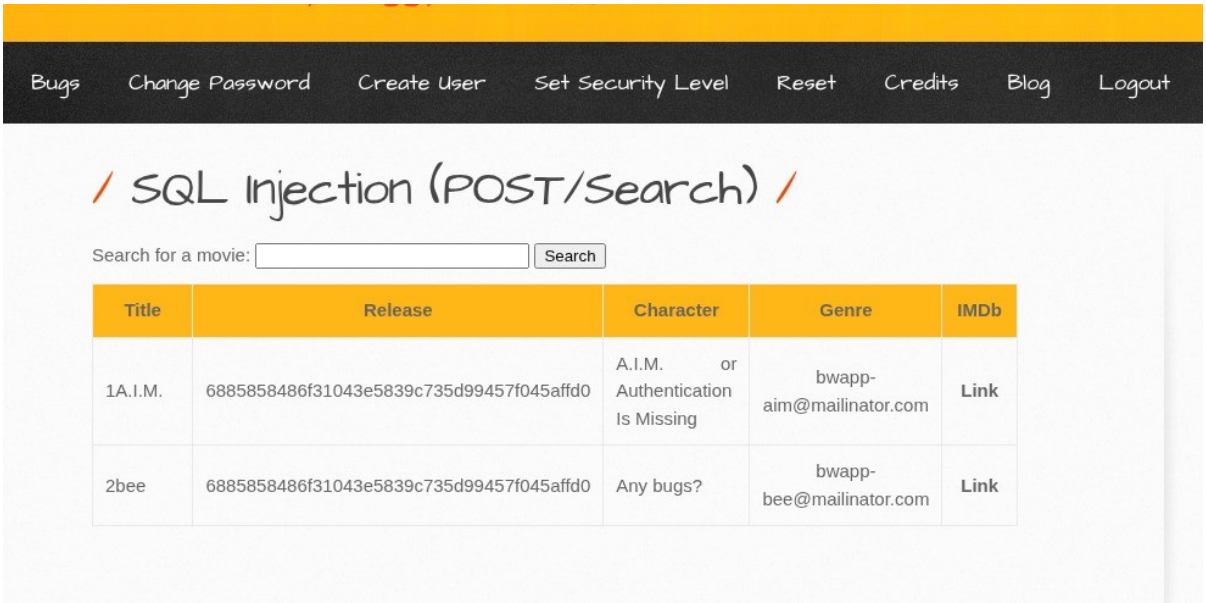
bWAPP SQL Injection page / baseline



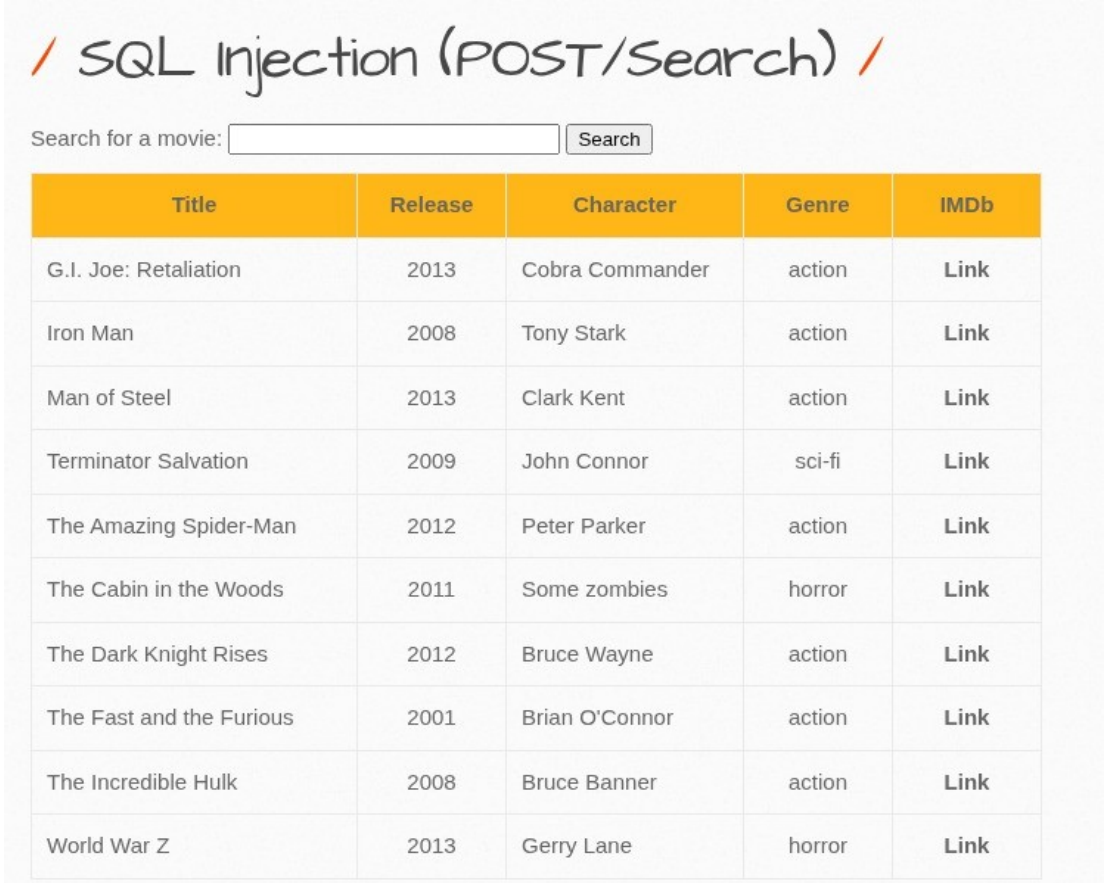
Baseline request evidence



Burp Repeater evidence



UNION-based injection request



Additional SQL Injection evidence

Impact

The evidence demonstrates unauthorised extraction of user-table information, including login values, password hashes and email addresses. The impact represents a major confidentiality and integrity risk.

Remediation

1. Use prepared statements / parameterised queries for all database operations.
2. Apply strict server-side validation to search parameters.
3. Run the database account with least privilege.
4. Disable verbose database errors in production.
5. Use a WAF only as defence-in-depth, not as a replacement for secure queries.

6.2 Remote & Local File Inclusion (RFI/LFI)

Finding ID	F-02
Severity	**Critical — CVSS 3.1: 9.1**
Location	rlfi.php — language parameter
Tester	Mohammed Ashraf Mokhtar
OWASP Mapping	WSTG Input Validation Testing — Testing for File Inclusion

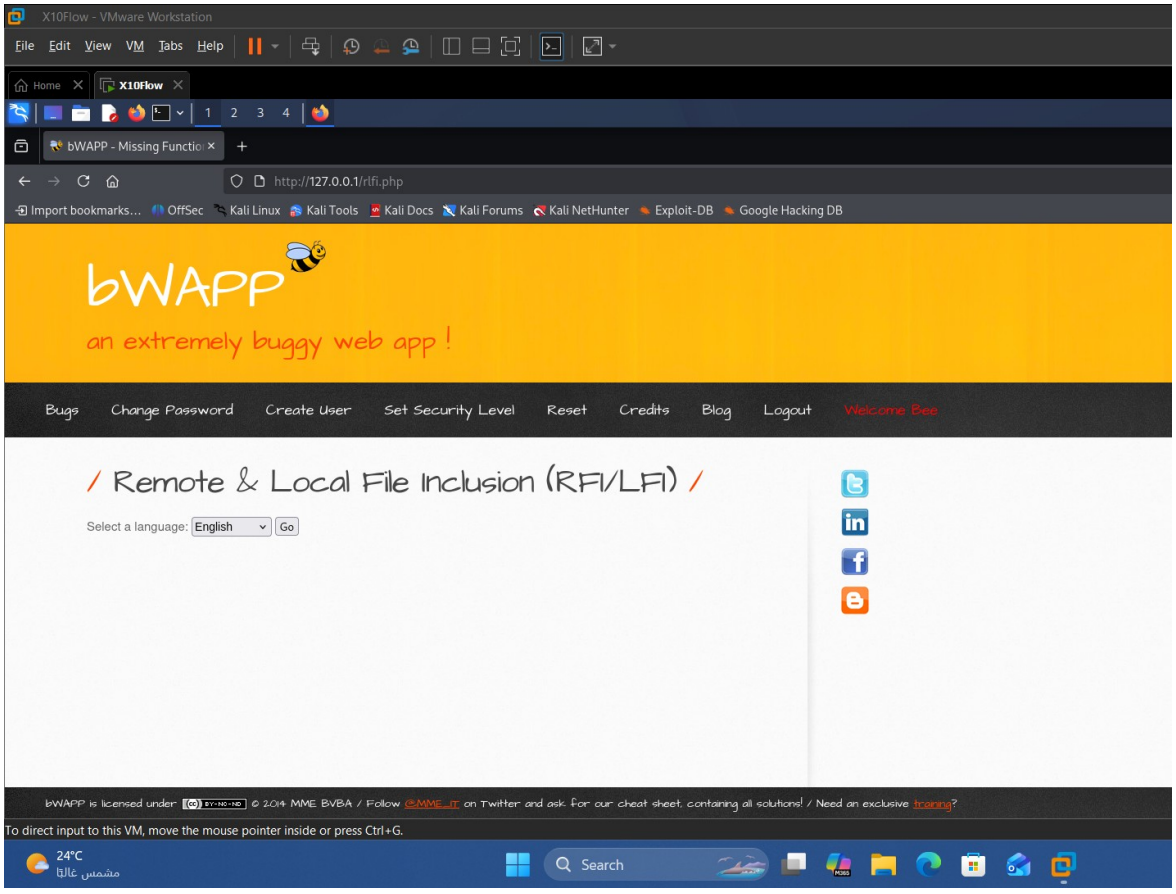
Description

The language parameter is used to select included content without sufficiently restricting the value to approved files. The documented tests demonstrate both remote content inclusion and local file disclosure through traversal.

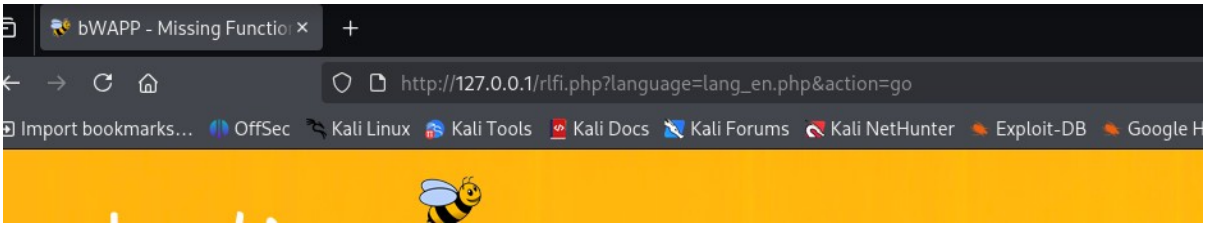
Steps to Reproduce

1. Confirm legitimate behaviour with language=lang_en.php.
2. Test remote inclusion using an external URL as the language value.
3. Test local inclusion using directory traversal: GET /rlfi.php?language=../../etc/passwd&action=go.
4. Observe remote content being rendered and /etc/passwd being disclosed.

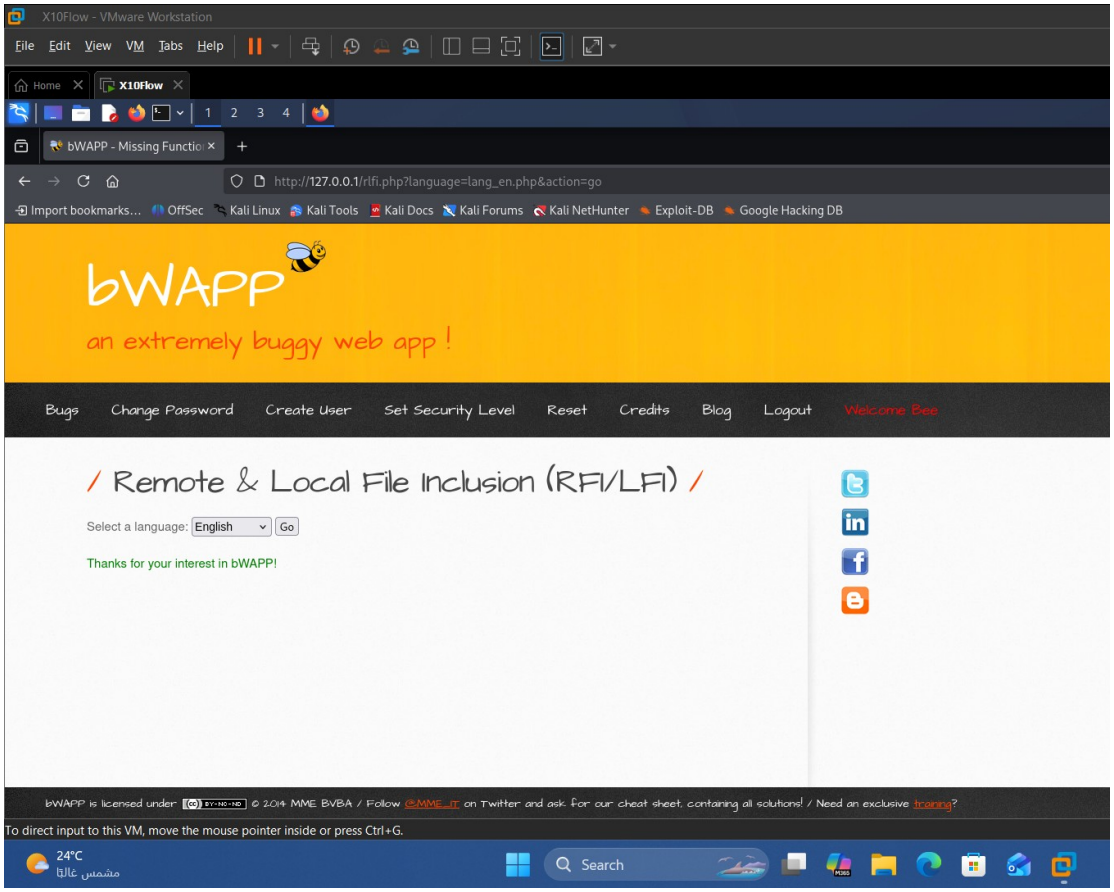
Evidence



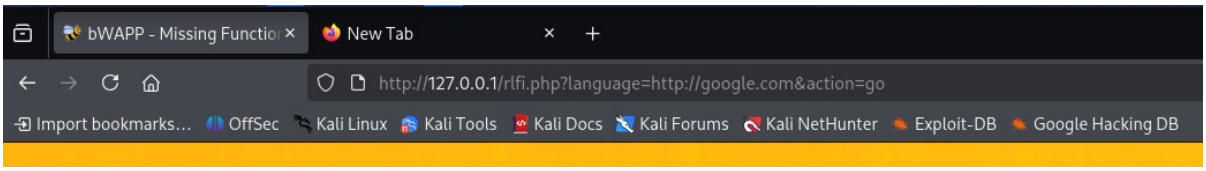
RFI/LFI baseline page



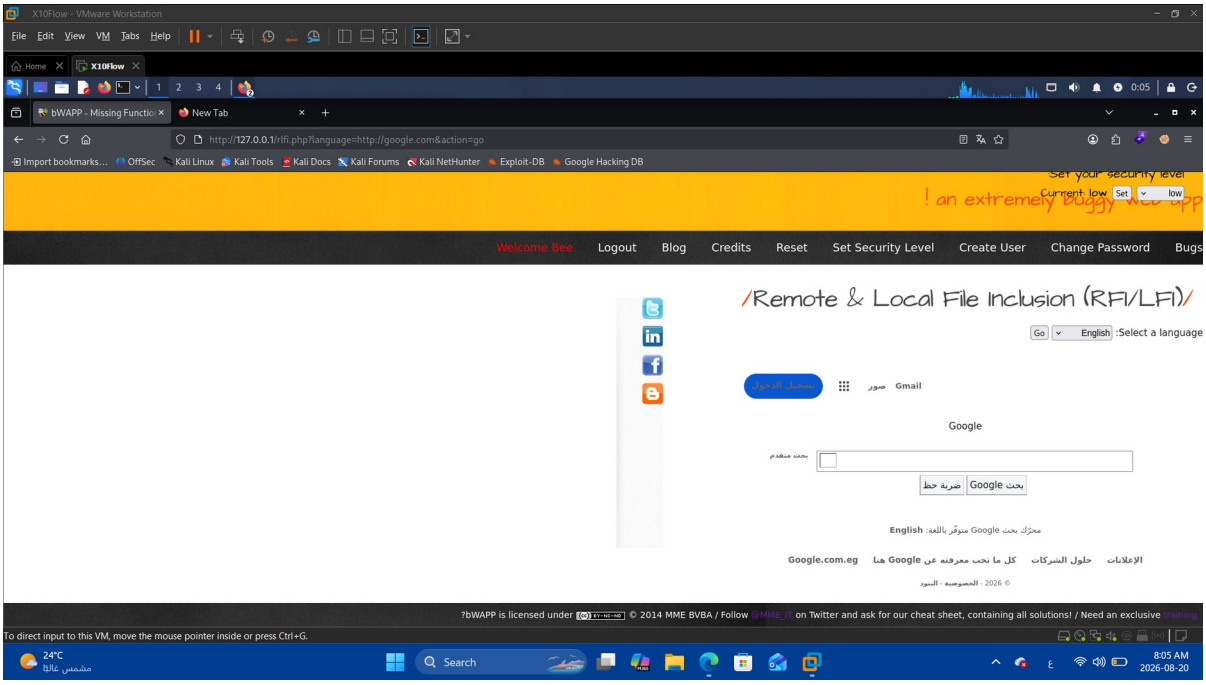
Legitimate language request



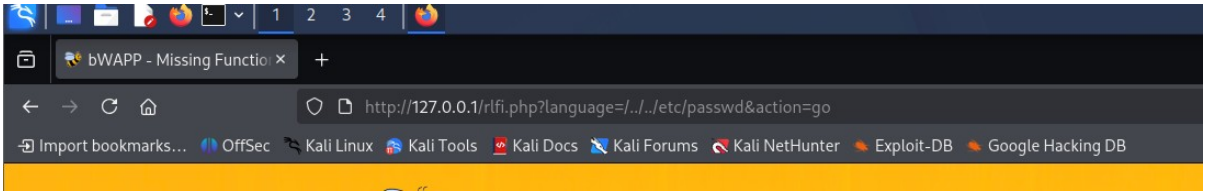
Expected application response



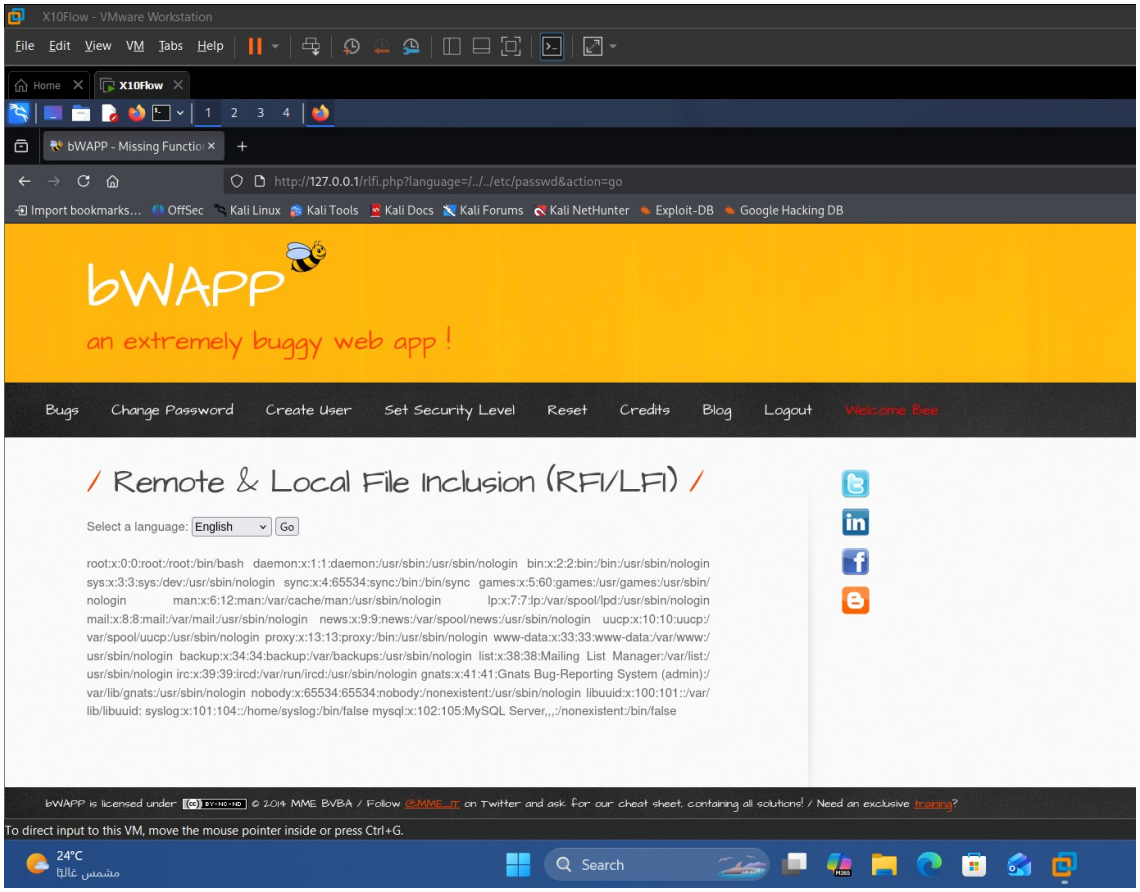
Remote inclusion test



Remote content rendered



Local traversal request



/etc/passwd disclosure

Impact

LFI can expose sensitive local files such as configuration files, credentials or source code. RFI can potentially allow malicious remote content to be included and, in vulnerable configurations, may lead to code execution.

Remediation

1. Never pass raw user input directly into include/require operations.
2. Use a strict server-side allow-list mapping accepted language values to known files.
3. Reject traversal sequences and canonicalise paths.
4. Disable remote file inclusion features where they are not required.
5. Apply least-privilege filesystem permissions to the web server.

6.3 Reflected Cross-Site Scripting — PHP_SELF / GET

Finding ID	F-03
Severity	**High — CVSS 3.1: 7.4**
Location	xss_php_self.php — firstname / lastname
Tester	Mohammed Ashraf Mokhtar
OWASP Mapping	WSTG Input Validation Testing — Testing for Reflected Cross-Site Scripting (WSTG-INPV-01)

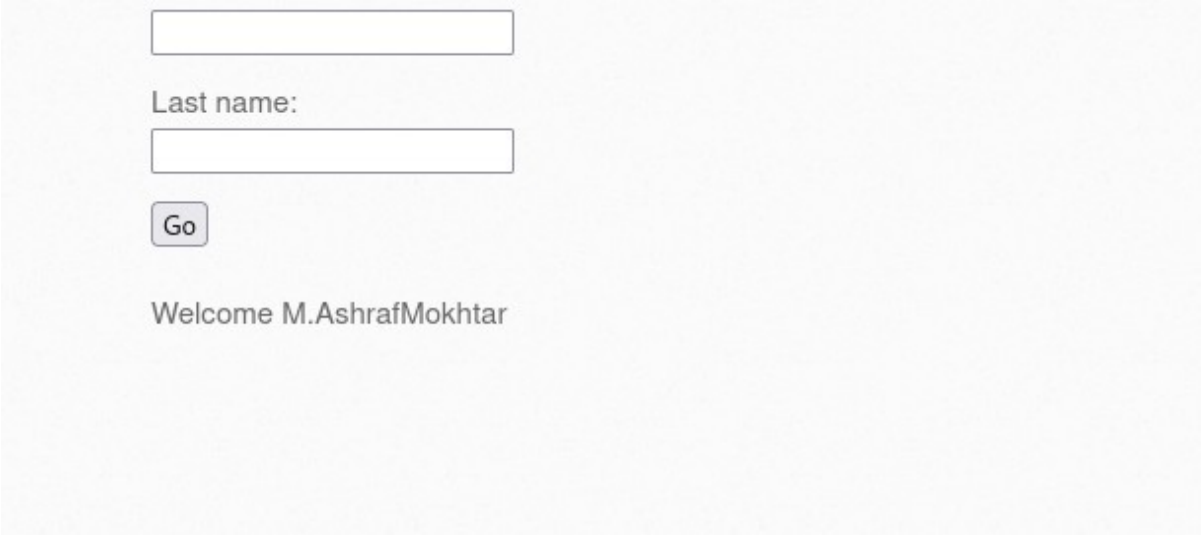
Description

The application reflects attacker-controlled input into the response without contextual output encoding. The PHP_SELF-based page accepts a crafted value that results in JavaScript execution in the browser.

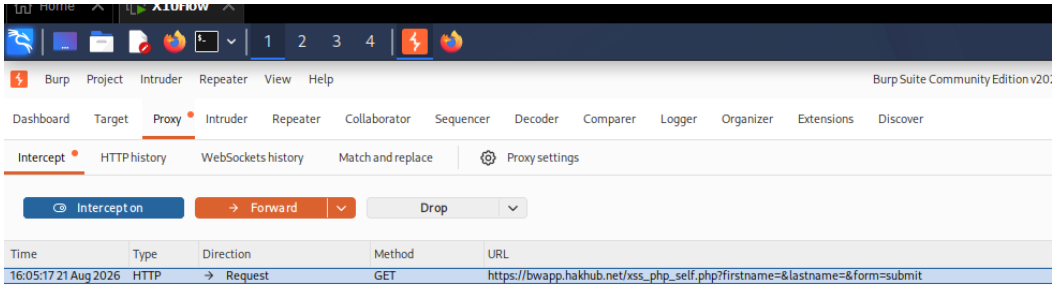
Steps to Reproduce

1. Open the reflected PHP_SELF XSS page and submit normal values.
2. Capture the request and identify the reflected firstname/lastname parameters.
3. Inject the documented script payload into the parameter.
4. Example: <script>alert(document.cookie)</script>.
5. Observe browser-side JavaScript execution.

Evidence

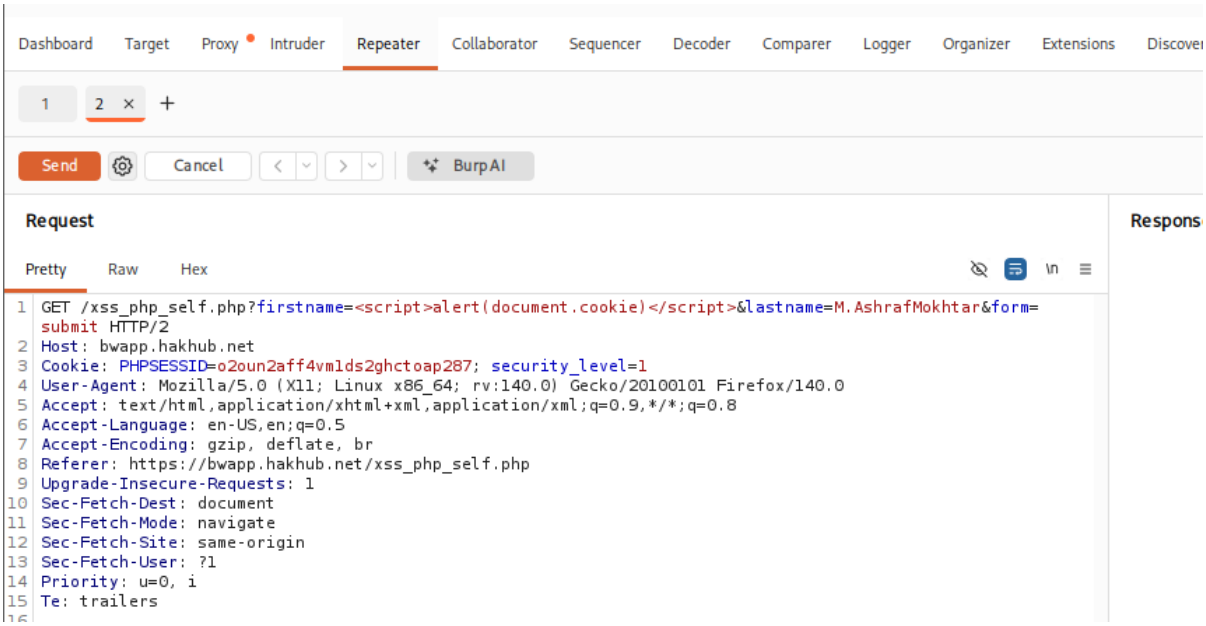


XSS PHP_SELF baseline

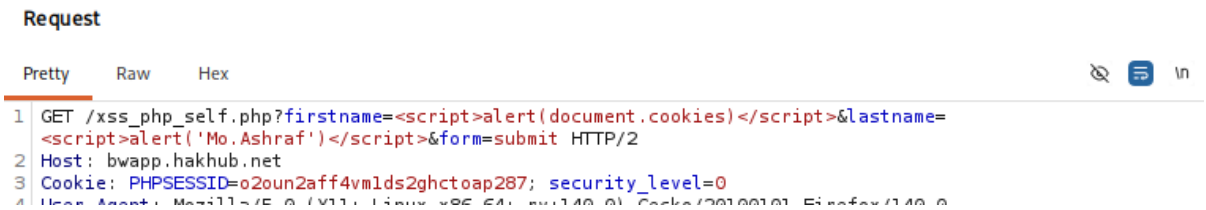




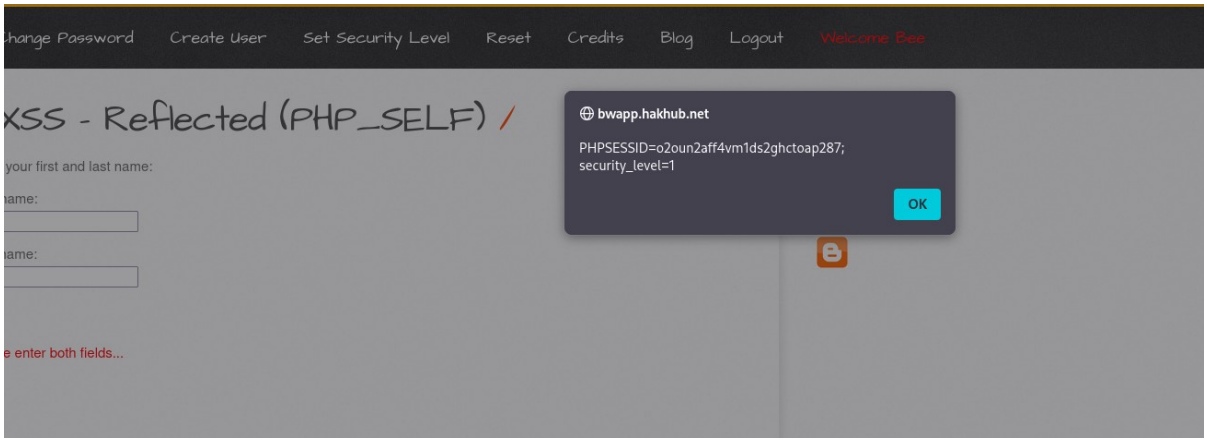
Captured request



Injected payload



Browser execution evidence



Additional XSS evidence

Impact

Reflected XSS can allow session-cookie exposure, phishing, page manipulation, malicious redirects and actions performed in the context of the victim's browser.

Remediation

1. Apply contextual output encoding before reflecting user input.
2. Use a strict Content Security Policy.
3. Set session cookies with HttpOnly and Secure attributes.
4. Validate input server-side and avoid unsafe reflection into HTML/JavaScript contexts.

6.4 Reflected Cross-Site Scripting — POST

Finding ID	F-04
Severity	**Medium — CVSS 3.1: 6.1**
Location	xss_post.php — firstname / lastname
Tester	Rana Ayman
OWASP Mapping	WSTG Input Validation Testing — Testing for Reflected Cross-Site Scripting (WSTG-INPV-01)

Description The website takes what you type into the form and shows it back on the page. Because it doesn't clean or filter that input, attackers can insert harmful code (like JavaScript). When the page loads, the browser runs that code. This means someone could trick the site into showing pop-ups, stealing cookies, or changing how the page looks and behaves.

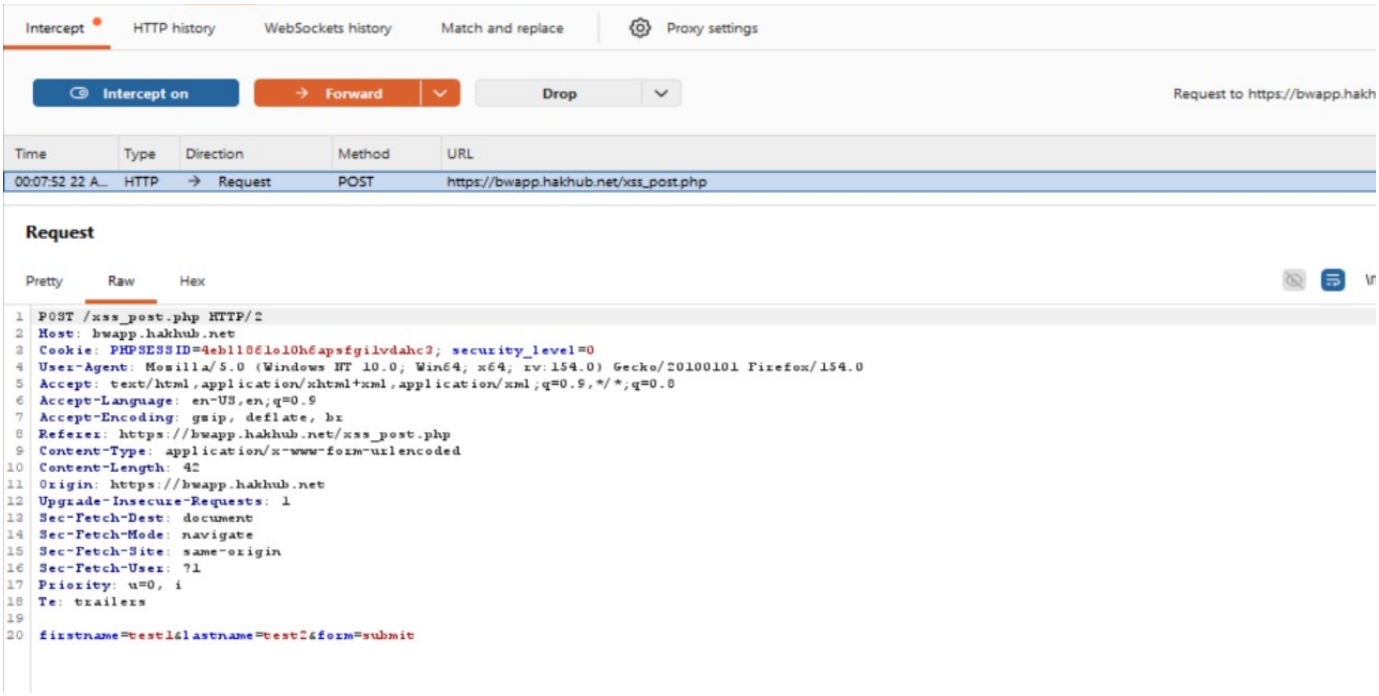
1 – Vulnerable form

The form is filled with harmless values (test1 and test2) to show how normal input is displayed back on the page before any attack.



POST XSS form

2 – Intercepted POST request in Burp Suite Capture the request that sends the form data (`firstname=test1`, `lastname=test2`, and `form=submit`) to the server. This shows how the browser transmits the user's input before any modification or payload injection.

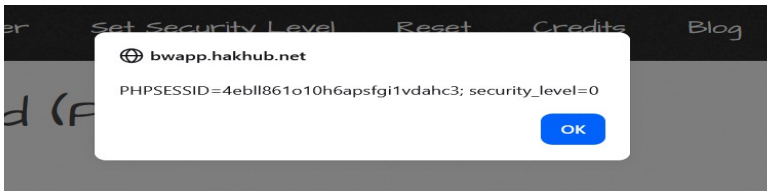


3 – Modified request with injected payloads Edit the intercepted request and replace the normal values (test1, test2) with malicious code. For example: firstname=<script>alert(document.cookie)</script> lastname=<script>alert("hacked")</script>

This demonstrates how an attacker can insert harmful JavaScript into the form fields.



4 – Alert box displaying PHPSESSID cookie Forward the modified request. The browser immediately runs the injected JavaScript and shows an alert containing the session cookie (PHPSESSID). This proves that the code was executed successfully.



5 – Alert box displaying custom message "hacked" Forwarding the modified request also triggers the second injected script. The browser displays an alert with the message "hacked," confirming that both payloads executed successfully.



Impact

- *Attackers could:*
- *Steal cookies or sensitive data from users.*
- *Trick users with fake pop-ups or redirects.*
- *Change how the page looks or behaves.*
- *Perform actions in the user's session without permission.*

Remediation

- **Encode or escape user input before showing it on the page.**
- **Validate input on the server side.**
- **Add Content Security Policy (CSP) headers to block unauthorized scripts.**
- Use frameworks that automatically protect against XSS.

6.5 IDOR / Client-Side Price Tampering

Finding ID	F-05
Severity	**High — CVSS 3.1: 8.2**
Location	insecure_direct_object_ref_2.php — ticket_quantity / ticket_price
Tester	Mohammed Ashraf Abdelwahab
OWASP Mapping	WSTG Authorization Testing + Business Logic Testing

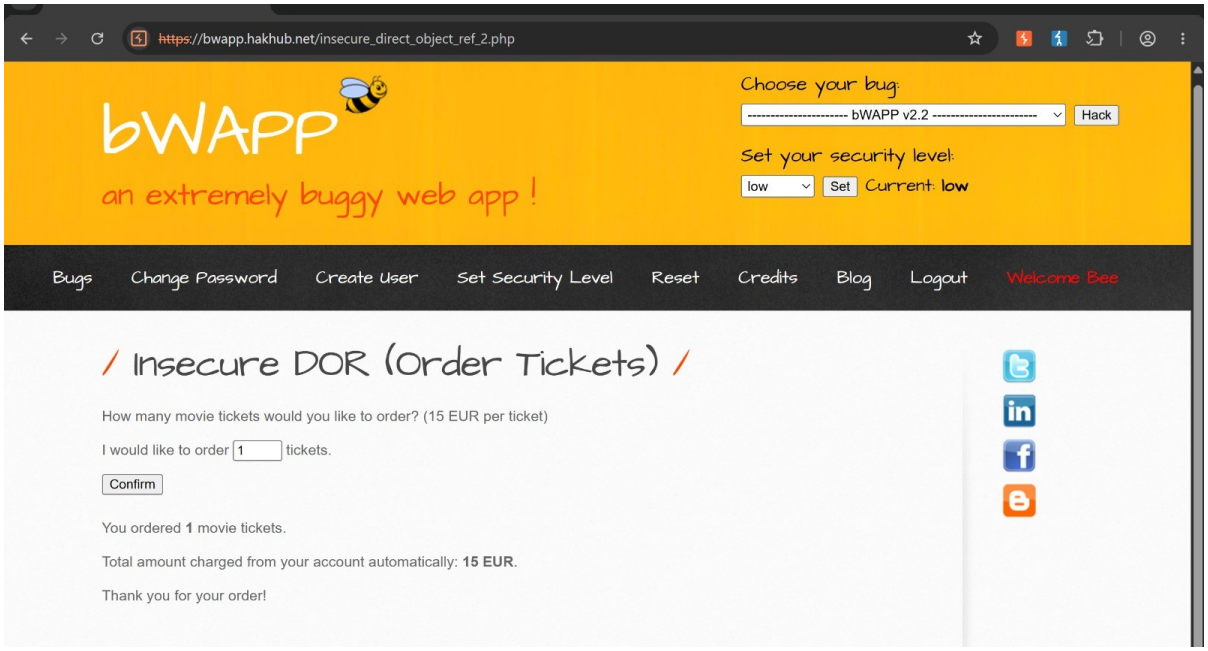
Description

The ticket-ordering functionality accepts a client-supplied ticket_price value instead of calculating the authoritative price on the server. The documented request modification changes the price from 15 to 0 while the order is still accepted.

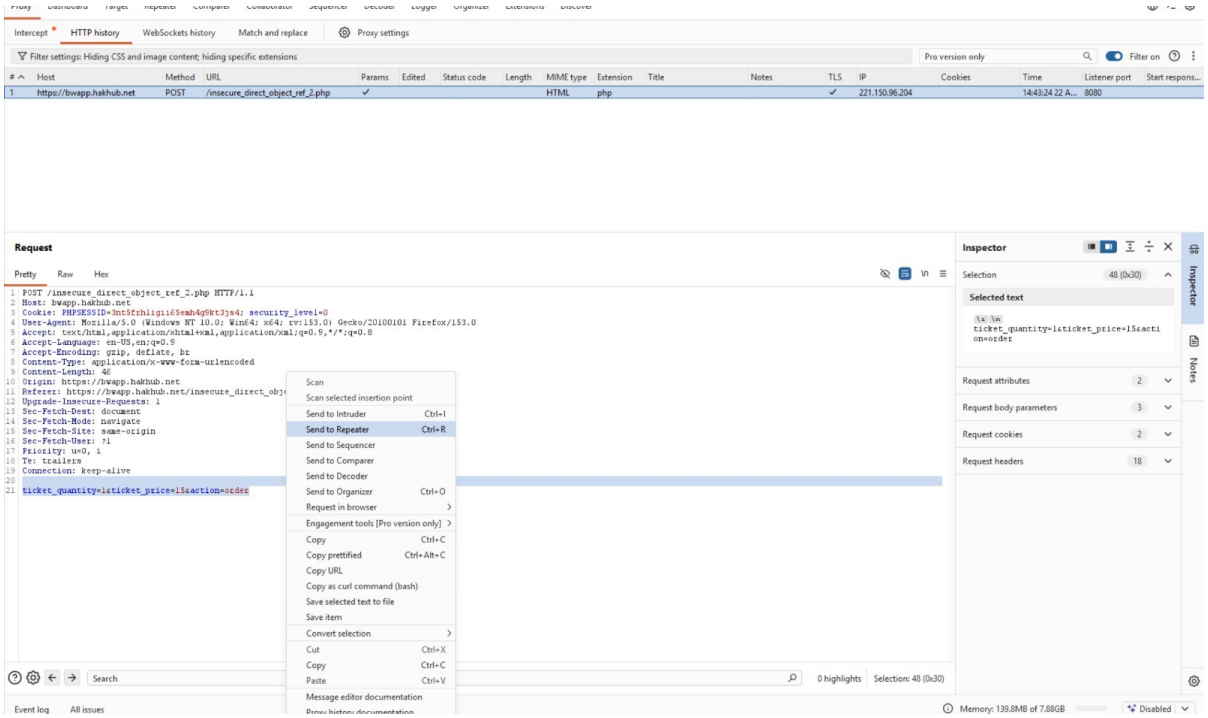
Steps to Reproduce

1. Place a normal order for one ticket and observe the expected price.
2. Intercept the POST order request in Burp Suite and send it to Repeater.
3. Baseline: ticket_quantity=1&ticket_price=15&action=order.
4. Tamper with the price: ticket_quantity=2&ticket_price=0&action=order.
5. Forward the modified request and observe acceptance of the manipulated amount.

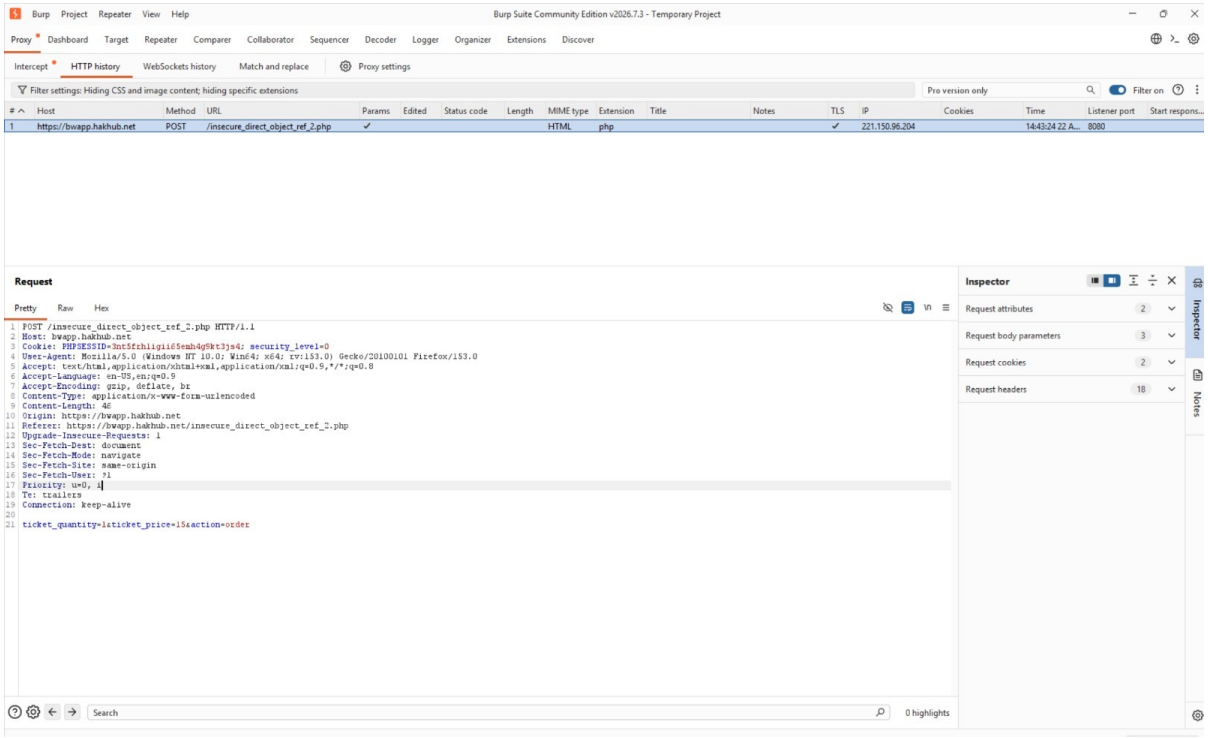
Evidence



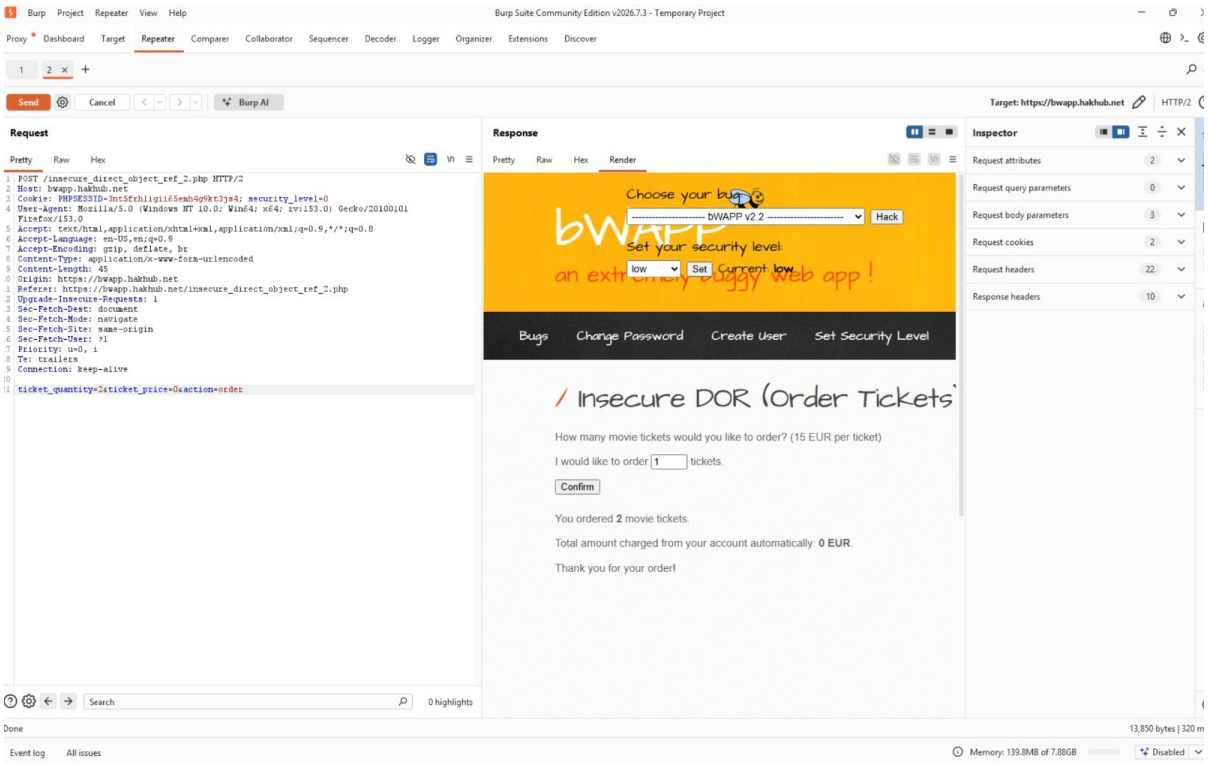
Ticket-ordering page



Normal order request



Tampered request



Response confirming manipulated price

Impact

An attacker can manipulate a business-critical monetary value controlled by the client. This can result in financial loss, fraud and inconsistent transaction records.

Remediation

1. Calculate price and total values exclusively on the server using trusted product data.
2. Accept quantity only as client input and validate its range.
3. Perform server-side authorization checks for every object reference.
4. Log and alert on mismatches between client-supplied and server-calculated amounts.

6.6 Reflected HTML Injection — GET

Finding ID	F-06
Severity	**Medium — CVSS 3.1: 5.4**
Location	htmli_get.php — firstname / lastname
Tester	Mohamed Ahmed Albasiony
OWASP Mapping	WSTG Input Validation Testing

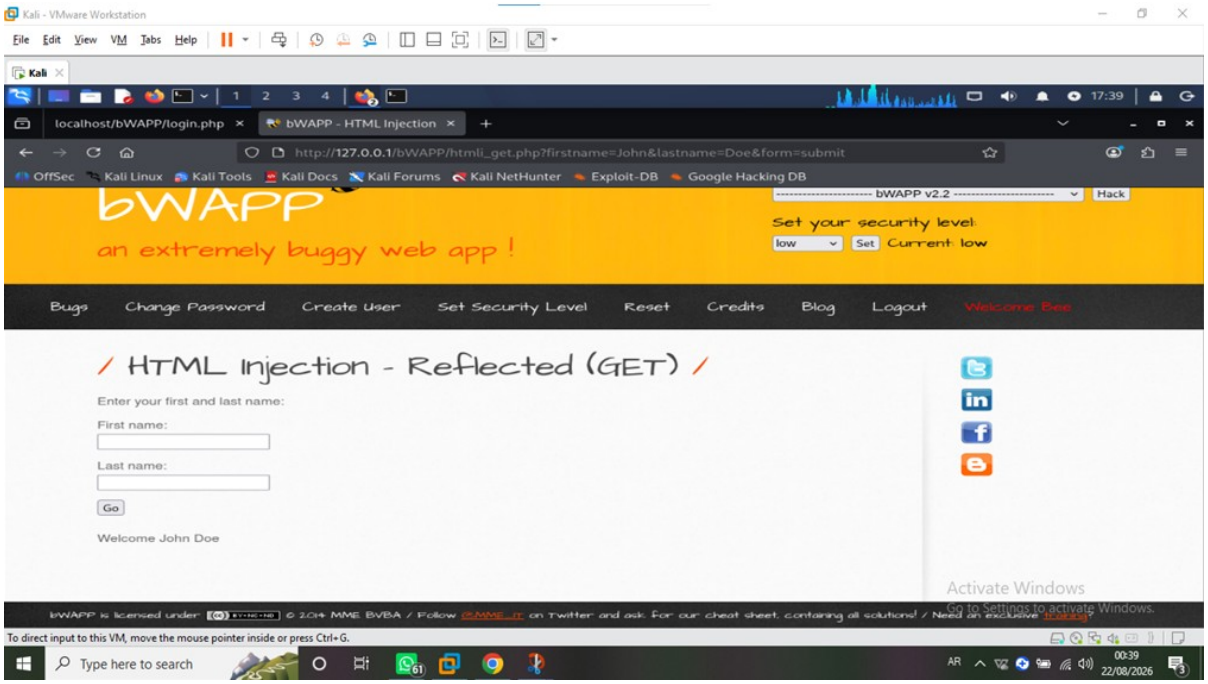
Description

The firstname and lastname fields are reflected into the response without HTML encoding, allowing arbitrary HTML markup to be rendered in the page.

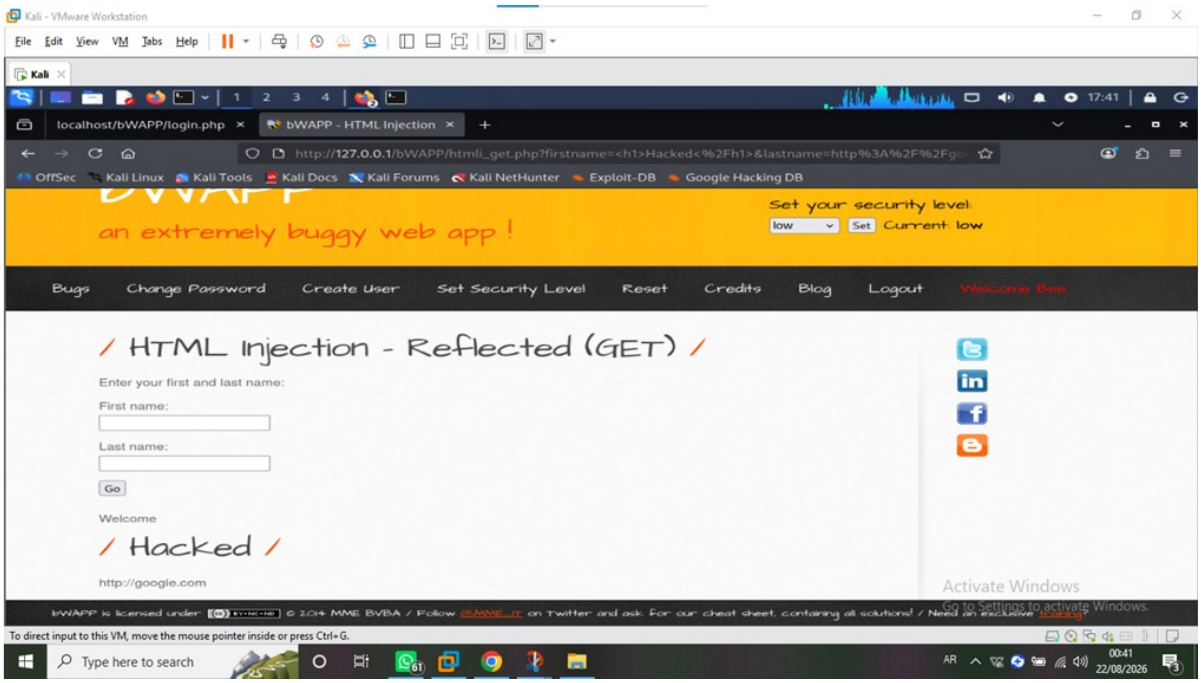
Steps to Reproduce

1. Submit normal values and confirm they are reflected as plain text.
2. Replace the firstname value with an HTML element.
3. Example: <h1>Hacked</h1>.
4. Observe the injected heading and link rendered directly in the application.

Evidence



HTML Injection baseline



Injected HTML rendered in the page

Impact

HTML injection can enable page defacement, fake content, phishing forms and misleading links. It becomes more serious if combined with a script-execution weakness.

Remediation

1. HTML-encode all user-controlled output.
2. Apply contextual output encoding consistently.
3. Use CSP as defence-in-depth.
4. Avoid reflecting raw user input where it is not required.

7. Risk Assessment

Risk was assessed using the severity ratings documented in the team's original bWAPP findings. CVSS values are retained where they were already assigned in the report; they are presented as the project's severity assessment rather than as a substitute for business-context risk analysis.

Severity	Count	Findings	Primary Impact
Critical	2	F-01, F-02	Database information exposure; arbitrary local file disclosure / potential code execution
High	2	F-03, F-05	Browser/session compromise; financial/business-logic manipulation
Medium	2	F-04, F-06	Reflected script/content injection

7.1 Common Root Causes

- Insufficient server-side validation of user-controlled input.
- Unsafe reflection of input into HTML responses.
- Direct use of client-controlled values in security-sensitive operations.
- Insufficient restrictions around file inclusion.
- Missing parameterisation for database queries.

8. Recommendations

1. Adopt secure input-handling standards across the application rather than patching each vulnerable parameter individually.
2. Use parameterised queries / prepared statements for all database access.
3. Implement allow-lists for file selection and eliminate direct user-controlled include paths.
4. Apply contextual output encoding to every reflected value.
5. Deploy a restrictive Content Security Policy and secure session-cookie attributes.
6. Perform all authorization and business-critical calculations on the server.
7. Apply least privilege to database and filesystem accounts.
8. Disable unnecessary verbose error information in production.
9. Repeat security testing after remediation and maintain regression tests for the vulnerable endpoints.

8.1 Recommended Retest Procedure

1. Re-run the original baseline requests.
2. Replay each original proof-of-concept payload.
3. Verify that the malicious input is rejected, encoded, or rendered harmlessly.
4. Confirm that server-side authorization and price calculations cannot be overridden by client parameters.
5. Confirm that sensitive files and database records are no longer exposed.

9. Conclusion

The bWAPP assessment demonstrated a complete web-application penetration-testing workflow rather than isolated vulnerability exploitation. The team began by identifying the application attack surface and candidate input points, proceeded to focused vulnerability identification and manual validation, and then documented impact and remediation.

The assessment confirmed six vulnerabilities across injection, file inclusion, client-side trust, authorization/business logic and reflected-output categories. The results demonstrate the value of combining a structured assessment process with application-specific manual testing.

For the graduation project, OWASP WSTG provides the primary web-security testing structure, while NIST SP 800-115 provides the broader technical testing and assessment process. This combination gives the report a clear methodology from reconnaissance through reporting.

10. References

OWASP Foundation — Web Security Testing Guide (WSTG).
https://owasp.org/www-project-web-security-testing-guide/

OWASP WSTG — Web Application Security Testing.
https://owasp.org/www-project-web-security-testing-guide/stable/4-Web_Application_Security_Testing/

OWASP WSTG — Testing for SQL Injection (WSTG-INPV-05).
https://wstg.owasp.org/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/05-Testing_for_SQL_Injection/

OWASP WSTG — Testing for Reflected Cross-Site Scripting (WSTG-INPV-01).
https://wstg.owasp.org/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/01-Testing_for_Reflected_Cross_Site_Scripting/

OWASP WSTG — Testing for File Inclusion.
https://wstg.owasp.org/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/11.1-Testing_for_File_Inclusion/

NIST SP 800-115 — Technical Guide to Information Security Testing and Assessment.
https://csrc.nist.gov/pubs/sp/800/115/final

10.1 Source Material Used for This Report

The vulnerability details, endpoints, team assignments, proof-of-concept evidence and original severity values were derived from the team's bWAPP Penetration Testing Report. The separate Final Web Application Penetration Testing report was used as the structural reference for organizing the engagement into methodology, reconnaissance, scanning, exploitation/validation, impact and reporting phases.

Where the bWAPP source did not contain evidence for a specific reconnaissance activity, this report deliberately avoids claiming that activity was performed.